
Oliver Krauss

Aggregation I

MDB3 | MTD.ba VZ WS26

Hagenberg 05./07.10.2026

HAGENBERG | LINZ | STEYR | WELS



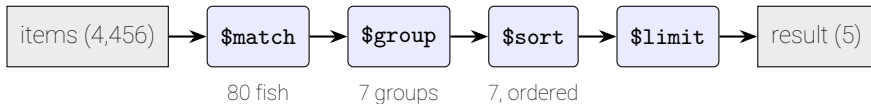
UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

GROUP BY, the Question Every Report Asks

```
SELECT type, COUNT(*) AS n, AVG(sell) AS avgSell
FROM items
GROUP BY type
ORDER BY n DESC
LIMIT 5;
```

- **find()** answers “which documents?”. It cannot answer “how many per type?” or “the average per group”
- In SQL that is **GROUP BY** with aggregate functions, and **HAVING** to filter the groups
- MongoDB has the same power, written as a **pipeline**: a list of stages the documents flow through

The Pipeline Model



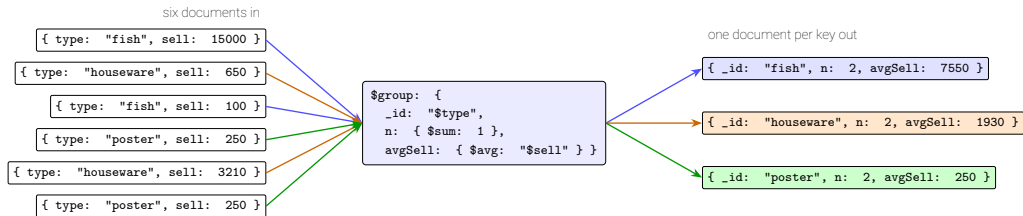
- Every stage takes documents in and hands documents out. The output of one stage is the input of the next
- **Order matters:** a `$match` before the `$group` filters rows, after it filters groups
- Stages are documents too: `{ $match: { ... } }`. The pipeline is an array of them:
`aggregate([ ... ])`

SQL Clauses, Pipeline Stages

SQL	Stage	Note
WHERE	\$match	same filter syntax as find
GROUP BY	\$group	_id is the group key
COUNT, SUM, AVG, MIN, MAX	accumulators in \$group	\$sum , \$avg , \$min , \$max
HAVING	\$match after \$group	the same stage, later
SELECT a, b AS c, expr	\$project	include, rename, compute
ORDER BY	\$sort	1 / -1
LIMIT / OFFSET	\$limit / \$skip	
SELECT COUNT(*)	\$count	one document with one number

-- SQL forces one order: **WHERE**, then **GROUP BY**, then **HAVING**, then **ORDER BY**. The pipeline lets you write the order you mean, and repeat stages

\$group, as a Picture



- Every input document lands in the group whose key equals its **type**. Same key, same output document
- The accumulators run **inside** each group: **\$sum: 1** counts its members, **\$avg** averages their **sell**
- Out comes one document per distinct key, and nothing else: **sell**, **name**, every other field is gone

\$group: GROUP BY with Accumulators

```
db.items.aggregate([
  { $group: { _id: "$type", n: { $sum: 1 }, avgSell: { $avg: "$sell" } } },
  { $sort: { n: -1 } },
  { $limit: 5 }
])
// { _id: 'houseware', n: 532, avgSell: 5274.55 }
// { _id: 'poster',    n: 452, avgSell: 250 }
// { _id: 'photo',     n: 391, avgSell: 10 }
// { _id: 'misc',      n: 383, avgSell: 5076.64 } ...
```

- **_id** is the **group key**. "\$type" with the dollar sign reads the field's value; "type" would be the string
- Every other field is an **accumulator**. \$sum: 1 counts, \$avg: "\$sell" averages the field
- The output has only **_id** and the accumulators. Everything else is gone after \$group

Accumulators

<code>\$sum: 1</code>	<code>COUNT(*)</code>
<code>\$sum: "\$sell"</code>	<code>SUM(sell)</code>
<code>\$avg \$min \$max</code>	<code>AVG MIN MAX</code>
<code>\$first \$last</code>	first / last seen
<code>\$push: "\$name"</code>	all values, as array
<code>\$addToSet</code>	distinct values

```
db.items.aggregate([
  { $match: { type: "fish" } },
  { $group: { _id: "$whereHow",
              n: { $sum: 1 },
              maxSell: { $max: "$sell" } } },
  { $sort: { maxSell: -1, n: -1 } }
])
// { _id: 'Sea',    n: 29, maxSell: 15000 }
// { _id: 'River', n: 27, maxSell: 15000 }
// { _id: 'River (cliff-top)', n: 4, ... } ...
```

- `$push` and `$addToSet` have no SQL equivalent: a group can carry a **list** of its members
- `$first` and `$last` only mean something after a `$sort`; otherwise the order is whatever came in

HAVING is a \$match After the \$group

```
SELECT type, AVG(sell) AS avgSell FROM items
GROUP BY type HAVING AVG(sell) > 5000 ORDER BY avgSell DESC;
```

```
db.items.aggregate([
  { $group: { _id: "$type", avgSell: { $avg: "$sell" } } },
  { $match: { avgSell: { $gt: 5000 } } },
  { $sort: { avgSell: -1 } }
])
// { _id: 'houseware', avgSell: 5274.55 } { _id: 'misc', avgSell: 5076.64 }
```

- After **\$group** the documents **are** the groups, so a **\$match** filters groups. No new keyword needed
- The filter names the accumulator's output field, **avgSell**, not **AVG(sell)**
- A **\$match** on **type** would have to come *before* the group: after it, **type** no longer exists

Group by Two Fields, or by Nothing

```
db.villagers.aggregate([
  { $group: { _id: { species: "$species", personality: "$personality" },
              n: { $sum: 1 } } },
  { $sort: { n: -1 } }, { $limit: 3 }
])
// { _id: { species: 'Rabbit', personality: 'Peppy' }, n: 8 }
// { _id: { species: 'Dog', personality: 'Lazy' }, n: 6 } ...

db.items.aggregate([ { $group: { _id: null, n: { $sum: 1 }, avg: { $avg: "$sell" } }
                      } ])
// { _id: null, n: 4456, avg: 1905.45 }
```

- GROUP BY **species, personality**: the key is a **document** with two fields
- **_id: null**: one group for everything, the aggregate function without **GROUP BY**
- The key can also be an expression: **_id: "\$birthday.month"**, or a computed bucket (*Aggregation II*)

Predict the Output

Your turn. What comes out, and how many documents? Shape first, values second.

```
// 1
db.villagers.aggregate([
  { $match: { species: "Octopus" } },
  { $group: { _id: "$species", names: { $push: "$name" }, n: { $sum: 1 } } }
])

// 2
db.items.aggregate([
  { $match: { type: "houseware", diy: true } },
  { $count: "diyHousewares" }
])

// 3
db.items.aggregate([
  { $group: { _id: "$type", n: { $sum: 1 } } },
  { $match: { type: "fish" } }
])
```

Predict the Output: Answers

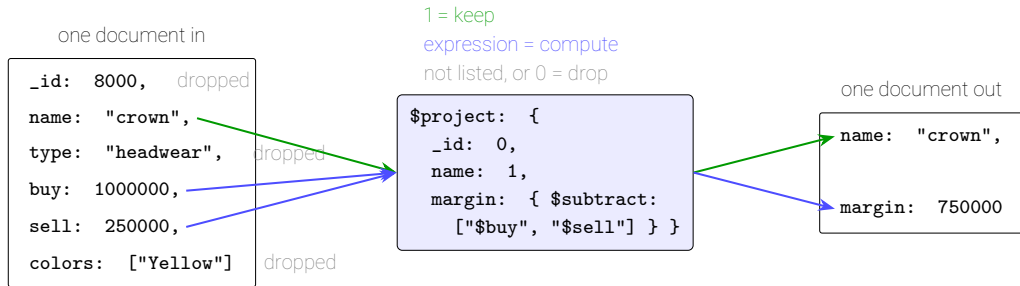
```
// 1: one document, because all octopuses share the group key
{ _id: 'Octopus', names: [ 'Marina', 'Octavian', 'Zucker' ], n: 3 }
```

```
// 2: one document with one field; $count names it
{ diyHousewares: 235 }
```

```
// 3: nothing. After $group there is no field "type", only _id and n.
//     The filter matches no document. No error, no warning, no result.
```

- Number 3 is the most common aggregation bug: a stage refers to a field that an earlier stage removed
- The fix is to run the pipeline **stage by stage** and look at what comes out. Compass does exactly that

\$project, as a Picture



- **\$project** works on **one document at a time**: the count in equals the count out
- A field is kept (1), dropped (absent, or `_id: 0`), or **computed** from the document's own fields by an expression
- That is the difference to **\$group**: no key, no collapsing, no accumulator. Just a new shape per document

\$project: SELECT with Expressions

```
db.items.aggregate([
  { $match: { buy: { $exists: true } } },
  { $project: { _id: 0, name: 1, margin: { $subtract: ["$buy", "$sell"] } } },
  { $sort: { margin: -1 } }, { $limit: 3 }
])
// { name: 'royal crown', margin: 900000 } { name: 'crown', margin: 750000 } ...

db.villagers.aggregate([
  { $project: { _id: 0, name: 1, owns: { $size: "$furniture" } } },
  { $sort: { owns: -1, name: 1 } }, { $limit: 3 }
])
// { name: 'Julian', owns: 26 } { name: 'Eunice', owns: 25 } ...
```

- 1 / 0 include or exclude, as in **find**. A new name with an **expression** computes a field
- Expressions (**\$subtract**, **\$concat**, **\$size**, ...) work per document. Accumulators work per group

The Whole Pipeline

```
db.villagers.aggregate([
  { $match: { "birthday.month": 10 } },
  { $group: { _id: "$birthday.day", villagers: { $push: "$name" } } },
  { $project: { _id: 0, day: "$_id", villagers: 1, n: { $size: "$villagers" } } },
  { $sort: { day: 1 } },
  { $limit: 4 }
])
// { villagers: [ 'Boyd', 'Raymond' ], day: 1, n: 2 }
// { villagers: [ 'Diva' ], day: 2, n: 1 }
// { villagers: [ 'Big Top' ], day: 3, n: 1 } ...
```

- Top to bottom: **filter, group, shape, order, cut**. That order covers most reports
- **\$project** renames **_id** to **day** with **"\$_id"** and computes **n** from the array the group built
- In SQL: **STRING_AGG**, a date function, and a subquery for the count

How to Write a Pipeline That Works

- **Start from the question:** “*per what* do I want *which* numbers?” *Per what* is the `_id`, *which numbers* are the accumulators
- **\$match first.** It shrinks the input and it is the only stage that can use an index (*Indexes and Validation*)
- **One stage per line**, and run the pipeline after every stage you add. Look at the output before adding the next
- **\$project last**, or not at all: shaping early throws away fields that later stages might need
- **Name outputs like columns:** `n`, `avgSell`, `maxSell`. A `$group` output called `_id` is the one you did not choose
- Write the SQL as a comment above it. If you cannot say it in SQL, you probably do not know what you are asking yet

Compass Builds It Stage by Stage

- Compass, collection **items**, tab **Aggregations**: add a stage, pick **\$match**, type the filter, and the preview shows the documents that come out
- Each further stage shows its own preview: the answer to “what does the **\$group** see?” is one click up
- **Export to language** turns the finished pipeline into PHP (or Java, Python, ...). Paste it into your script, done
- The pipeline is the same document array, whatever built it. Compass is a debugger for pipelines, not a different language

aggregate from PHP

```
$perType = $items->aggregate([
    ['$group' => ['_id' => '$type', 'n' => ['$sum' => 1],
        'avgSell' => ['$avg' => '$sell']]],
    ['$sort' => ['n' => -1]],
    ['$limit' => 3],
]);
foreach ($perType as $row) {
    printf("%-10s %4d items, avg %8.1f Bells\n", $row->_id, $row->n, $row->avgSell);
}
// houseware    532 items, avg    5274.5 Bells
// poster       452 items, avg     250.0 Bells
// photo        391 items, avg      10.0 Bells
```

- The pipeline is an array of stage arrays; "**\$type**" stays a string, in single quotes
- **aggregate** returns a cursor like **find**; the rows are documents with the fields you named

Translate This SQL

With your neighbour. Write the pipeline. Then guess the first row.

```
-- 1
SELECT species, COUNT(*) AS n FROM villagers
GROUP BY species ORDER BY n DESC LIMIT 3;

-- 2
SELECT whereHow, MAX(sell) AS maxSell FROM items WHERE type = 'insect'
GROUP BY whereHow HAVING MAX(sell) >= 8000 ORDER BY maxSell DESC;

-- 3 (there is no SQL for this one without a junction table)
-- per personality: the average number of items a villager owns, top 3
```

Translate This SQL: Answers

```
// 1: Cat 23, Rabbit 20, Squirrel 18
```

```
[ { $group: { _id: "$species", n: { $sum: 1 } } },  
  { $sort: { n: -1 } }, { $limit: 3 } ]
```

```
// 2: On palm trees 12000, On trees (any kind) 10000, On the ground 8000
```

```
[ { $match: { type: "insect" } },  
  { $group: { _id: "$whereHow", maxSell: { $max: "$sell" } } },  
  { $match: { maxSell: { $gte: 8000 } } },  
  { $sort: { maxSell: -1 } } ]
```

```
// 3: Normal 13.37, Peppy 12.94, Smug 12.62. Per document first, then per group.
```

```
[ { $project: { personality: 1, owns: { $size: "$furniture" } } },  
  { $group: { _id: "$personality", avgOwns: { $avg: "$owns" } } },  
  { $sort: { avgOwns: -1 } }, { $limit: 3 } ]
```

Quick Retrieval: Test Yourself

Before the summary: can you answer these from memory?

1. What does a stage take in and hand out? Why does the order of stages matter?
2. In `$group`, what is `_id`? What is the difference between `"$type"` and `"type"`?
3. How do you write **HAVING**?
4. Which fields exist after a `$group`?
5. `$avg` and `$subtract`: which one goes into `$group`, which into `$project`? Why?
6. Why should `$match` be the first stage?

Think first, then we go through the answers.

Quick Retrieval: Answers

1. Documents in, documents out. Each stage sees only what the previous one emitted, so a **\$match** before and after a **\$group** filter different things.
2. The group key. "**\$type**" is the value of the field, "**type**" is the literal string.
3. A **\$match** after the **\$group**, on the accumulator's output field.
4. Only **_id** and the accumulators you named.
5. **\$avg** is an accumulator, per group; **\$subtract** is an expression, per document.
6. Fewer documents for every later stage, and it is the stage that can use an index.

Summary: Aggregation I

The model

- A pipeline is an array of stages; documents flow through in order
- `$match`, `$group`, `$sort`, `$limit`, `$project`, `$count` cover **GROUP BY** reports
- **HAVING** is `$match` after `$group`; two-field keys are documents; `_id: null` is the grand total

Next

- Arrays inside groups: `$unwind` turns **furniture** into one document per item (*Aggregation II*)
- Joins after all: `$lookup`, and why it is not a **JOIN**
- Buckets, facets, and writing results back with `$out` and `$merge`

Details: Memory Limits and allowDiskUse

```
db.items.aggregate([ ... ], { allowDiskUse: true })
```

- Every stage may use up to 100 MB of RAM; **\$group** and **\$sort** on a large input hit that limit first
- **allowDiskUse** lets those stages spill to temporary files. Slower, but it finishes
- A **\$sort** followed by a **\$limit** is optimised into a top-N: it keeps only **N** documents in memory
- Since MongoDB 6.0 the server allows disk use by default; the option still exists to turn it *off*

Details: explain

```
db.items.explain("executionStats").aggregate([
  { $match: { type: "fish" } },
  { $group: { _id: "$whereHow", n: { $sum: 1 } } }
])
// stages[0].$cursor.executionStats: totalDocsExamined, nReturned, stage: COLLSCAN
```

- The first **\$match** becomes the query of an internal cursor; **explain** shows whether it used an index or scanned everything
- The server also reorders stages where that is safe: a **\$match** on a group key moves in front of the **\$group**
- How to read the plan and when to add an index: *Indexes and Validation*

Details: More Accumulators

```
{ $group: { _id: "$type",  
  cheapest: { $first: "$name" },           // after a $sort on sell  
  distinctColors: { $addToSet: "$colors" }, // arrays of arrays here  
  spread: { $stdDevPop: "$sell" },  
  top: { $top: { sortBy: { sell: -1 }, output: "$name" } },  
  three: { $topN: { n: 3, sortBy: { sell: -1 }, output: "$name" } } } }
```

- **\$top**, **\$topN**, **\$bottom**, **\$firstN**, **\$minN** (5.2+) pick members without a separate sort stage
- **\$addToSet** on an array field collects whole arrays; to collect the elements, **\$unwind** first (*Aggregation II*)
- **\$count** exists as an accumulator too (5.0+): **n: { \$count: {} }** is **\$sum: 1**

Details: Expression Operators

Arithmetic	<code>\$add \$subtract \$multiply \$divide \$mod \$round</code>
Strings	<code>\$concat \$toUpper \$toLower \$substrCP \$strLenCP \$split</code>
Arrays	<code>\$size \$arrayElemAt \$first \$last \$in \$filter \$map</code>
Dates	<code>\$year \$month \$dayOfMonth \$dateToString \$dateDiff</code>
Conditions	<code>\$cond \$ifNull \$switch</code>
Comparison	<code>\$eq \$gt \$gte ...</code> (return true/false, unlike the query operators)
Types	<code>\$toInt \$toString \$toDate \$type</code>

```
{ $project: { label: { $concat: ["$name", " (", "$type", ")"] },  
              pricey: { $cond: [ { $gt: ["$sell", 10000] }, "yes", "no" ] } } }
```

- Expressions can be used in `$project`, `$addField`s, the group key, and in `$match` via `$expr`

Details: \$addFields, \$set, \$unset, \$sortByCount

```
{ $addFields: { owns: { $size: "$furniture" } } } // keeps every other field
{ $set: { owns: { $size: "$furniture" } } } // the same stage, newer name
{ $unset: ["furniture", "colors"] } // drop fields, keep the rest

db.items.aggregate([ { $sortByCount: "$type" }, { $limit: 3 } ])
// { _id: 'houseware', count: 532 } { _id: 'poster', count: 452 } ...
```

- **\$project** is all-or-nothing: name a field and everything else disappears. **\$addFields** adds and keeps
- **\$sortByCount** is **\$group** with **\$sum: 1** plus **\$sort** descending, in one stage